

An indicator-based evolutionary algorithm with adaptive archive update cycle for multi-objective multi-robot task allocation

Chengxin Wen, Hongbin Ma^{*}

School of Automation, Beijing Institute of Technology, Beijing, 100081, China

ARTICLE INFO

Communicated by C. Yang

Keywords:

Multi-robot system
Multi-robot task allocation
Multi-objective optimization
Evolutionary algorithm

ABSTRACT

The multi-robot task allocation problem has garnered significant attention in research and development. This paper addresses the multi-robot task allocation problem by introducing a unified model that seamlessly transforms into four popular mathematical models through simple parameter adjustments. We propose an efficient indicator-based multi-objective evolutionary algorithm with a hybrid encoding scheme for task execution order and robot starting point information. The algorithm employs the hypervolume indicator for environmental selection to enhance convergence and the modified crowding distance for archive updates to promote diversity. Additionally, an adaptive archive update mechanism is designed for time efficiency. In experiments comparing our proposed algorithm with 8 state-of-the-art algorithms on 18 randomly generated instances of various sizes, along with 4 real-world problems from the TSPLIB benchmark, our algorithm consistently outperformed the comparison algorithms in five key indicators. These results underscore the effectiveness of our approach in addressing real-world multi-robot task allocation problems.

1. Introduction

In recent years, there has been a considerable focus among researchers on Multi-Robot Systems (MRS). The collaborative nature inherent in MRS enables the optimization of task execution efficiency by leveraging the collective capabilities of robots. Moreover, the precision actuators equipped on these robots empower them to undertake tasks requiring a high level of precision. This unique combination of collaborative potential and precision operation has rendered MRS applicable across a diverse array of fields, including but not limited to healthcare [1], anti-terrorism [2], disaster rescue [3], inspection [4], exploration [5], and routing [6]. The versatility of MRS highlights its potential to significantly enhance performance and outcomes in a wide range of applications.

Within the realm of MRS, the Multi-Robot Task Allocation (MRTA) problem emerges as a pivotal and complex challenge. This problem mainly considers two aspects, which the set of tasks each robot is assigned and the order in which they are executed. When multiple robots collaborate in a shared environment, efficient distribution of tasks among them becomes the cornerstone for optimal performance, resource utilization, and overall system efficiency. The complexity of tasking involves considerations of robot capabilities, environmental constraints, mission goals, and dynamic changes in the operating environment. Distinguished by their distinct mechanisms for addressing

MRTA problems, prevailing solutions can be broadly classified into two categories: market-based and optimization-based approaches [7].

The market-based method emulates market mechanisms for the allocation of tasks to individual robots. In this approach, an agent (acting as a simulated market) orchestrates the interaction between tasks and robots. Tasks are proposed and presented to robots, which make decisions based on their capabilities, current workload, and potential rewards. Task assignments may undergo multiple rounds of competition, negotiation, or auction until a resolution is reached. The strength of the market-based MRTA method lies in its simulation of real-market competition and negotiation processes, facilitating effective task allocation and resource utilization. However, this method places high demands on communication stability and entails a substantial communication load for agent robots.

The optimization-based method formalizes the MRTA into a mathematical model in order to find the optimal solution through an optimization algorithm. Factors such as tasks, robots, and environments are abstracted into mathematical variables and constraints. It defines an objective function, which is used to measure the quality of the task allocation plan. The objective function usually reflects indicators such as task completion time, resource utilization efficiency, cost, etc. The goal of optimization algorithms is to minimize or maximize this objective function. The optimization-based method aims to find the task

^{*} Corresponding author.

E-mail addresses: 3120205445@bit.edu.cn (C. Wen), mathmhb@bit.edu.cn (H. Ma).

<https://doi.org/10.1016/j.neucom.2024.127836>

Received 24 January 2024; Received in revised form 11 April 2024; Accepted 5 May 2024

Available online 10 May 2024

0925-2312/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

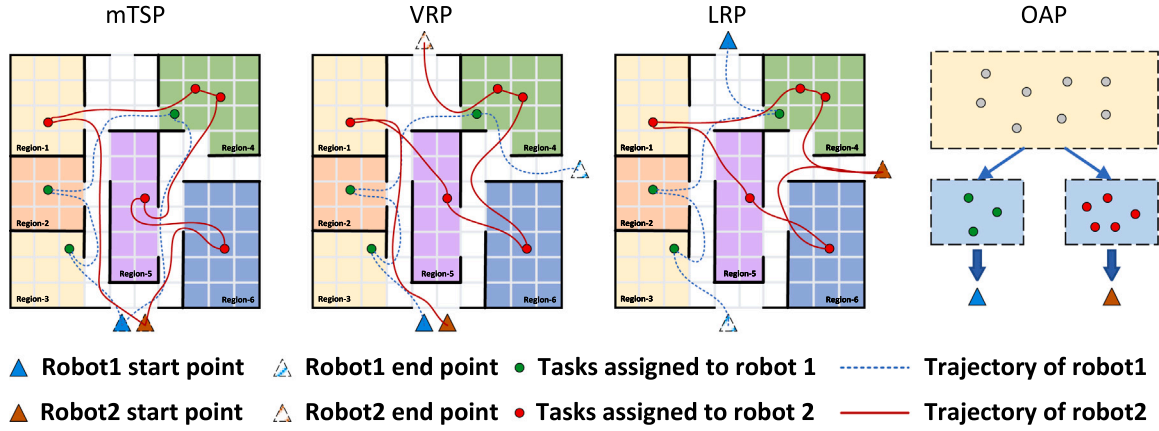


Fig. 1. Illustration of four main MRTA mathematical models.

allocation scheme with optimal overall system performance, which can globally optimize system benefits, and can be customized according to the needs and constraints of specific problems to adapt to different tasks and environments.

In the modeling of MRTA problems, diverse mathematical models can be formulated to capture distinct problem characteristics. Fig. 1 illustrates the principal mathematical models presently employed for representing MRTA problems. The multiple Traveling Salesman Problem (mTSP) stands out as the predominant mathematical model often adapted for MRTA. Within this model, all robots initiate their journeys from a common point and are tasked with returning to that specific location upon completing their assigned tasks. Comparable to the mTSP, the multiple Vehicle Routing Problem (mVRP) involves multiple starting and ending points. However, unlike the mTSP, each robot in the mVRP must commence from a designated starting point and ultimately return to a specified endpoint. The Location-Routing Problem (LRP) shares similarities with the mVRP, encompassing multiple starting and ending points. Yet, the LRP distinguishes itself by necessitating the planning of both the starting and ending points for each individual robot. In contrast, the Optimal Allocation Problem (OAP) focuses solely on determining which tasks are assigned to each robot. Unlike the other models, the OAP does not address the sequencing of task execution, concentrating solely on the optimal allocation of tasks to robots.

In essence, the aforementioned mathematical models effectively capture the nuances of Multi-Robot Task Allocation (MRTA) problems. Nevertheless, real-world MRTA scenarios often present uncertainties. As the problem characteristics evolve, researchers typically find themselves compelled to reconfigure mathematical models, adding a considerable burden to their workload. The above challenges motivated us to design a unified mathematical model and corresponding methods to solve the model. The main contributions of this work are divided in several folds.

- A unified optimization model is designed to represent the multi-objective MRTA problem. By adjusting the model parameters, the model can be transformed into four optimization models: mTSP, mVRP, LRP, and OAP.
- To effectively solve the proposed model, an indicator-based evolutionary algorithm is designed. The algorithm uses an external archive to save valuable solutions obtained during the search process and uses hypervolume for environmental selection and inverted generational distance for archive update.
- An adaptive archive update cycle mechanism is designed to improve the solution speed of the algorithm. It dynamically adjusts the algorithm search cycle according to the search phase of the algorithm.
- The proposed algorithm was compared with eight advanced evolutionary algorithms on 18 problems with different scales, and the

results showed the efficiency and effectiveness of the proposed algorithm in solving multi-objective MRTA problems.

The remainder of this paper is structured as follows: In Section 2, we provide a summary of related work. Section 3 outlines the system model and articulates the unified MRTA problem. The specifics of our proposed approach are presented in Section 4. Experimental studies are detailed in Section 5. In conclusion, Section 6 wraps up this paper.

2. Related works

In this article, the MRTA problem is modeled as an optimization problem. Therefore, this section mainly refers to articles that use optimization-based methods to solve MRTA.

The main purpose of the MRTA problem is to optimize certain indicators of MRS (e.g., task execution time, overall cost, etc.) by planning the task execution sequence of each robot [8,9]. To achieve this, a logical approach is to conceptualize the MRTA problem as an optimization problem. Initially, the MRTA problem was conceptualized as an optimal allocation problem [10], focusing solely on determining which tasks are assigned to each robot, without delving into the specific execution order of the robots [11]. Nam et al. [12] considered MRTA in an uncertain dynamic environment where the cost for computing task allocation is not completely known, and designed a heuristic algorithm to solve it. Liu et al. [13] devised a top-down approach tailored for large-scale task scenarios featuring dynamic characteristics. This approach aims to decentralize problems and localize information as much as possible. In the experimental phase, when contrasted with traditional methods, this approach demonstrated outstanding planning efficacy. To enhance planning efficiency, Zhang et al. [14] devised a learning-based approach to optimize task sequence allocation in the assembly process. In addressing MRTA and path planning challenges in the context of industrial plant inspection, Liu et al. [15] introduced a novel genetic algorithm that incorporates environmental representation. Through a simulation study conducted in a sizable industrial site, the algorithm yielded optimal results, showcasing its effectiveness.

To address the sequencing of tasks in robot execution, researchers approached MRTA by modeling it as a Multiple Traveling Salesman Problem. In their work, Msala et al. [16] took into account multiple performance indicators simultaneously, including efficiency, speed, and cost. They devised a hybrid optimization algorithm by combining the K-means clustering algorithm with a genetic algorithm. Wei et al. [17] introduced robot load balancing indicators to the conventional mTSP. They reformulated the problem as a multi-objective optimization challenge and put forth a novel multi-objective particle swarm optimization method to effectively address this enriched problem formulation. In addition, the MRTA problem is also modeled as a variant of TSP, such

Table 1
Summary and comparison of existing literature on MRTA.

Ref.	Problem model	Optimization objectives			Problem attribute	
		Execution time	Transfer time	Load balance	Heterogeneous robots	Multiple tasks
[12]	OAP	✓	–	–	✓	–
[13]	OAP	✓	✓	–	✓	✓
[15]	OAP	✓	✓	✓	–	✓
[16]	mTSP	✓	–	–	–	–
[17]	mTSP	✓	–	✓	–	–
[18]	mTSP	✓	✓	–	–	–
[20]	mVRP	✓	✓	–	–	✓
[21]	mVRP	✓	–	–	–	–
[22]	mVRP	✓	✓	–	–	–
[23]	mVRP	✓	✓	–	✓	✓
[24]	LRP	✓	–	–	–	–
[25]	LRP	✓	✓	–	✓	✓

as TSP-D [18] and TW-TSP [19], which increases the applicability of this problem model.

While the multi-traveling salesman problem enables the planning of robot task execution sequences, it overlooks the constraints specific to the robots involved. Consequently, researchers pivoted towards modeling the MRTA problem using a Vehicle Routing Problem model. This shift allowed for the inclusion of robot-specific constraints in the allocation and sequencing of tasks. Panchu et al. [20] enhanced the designed VRP model by incorporating task priority constraints. They further developed a variable parameter genetic algorithm to derive solutions that account for these additional constraints. Sarkar et al. [21] took into account the capacity differences among various robots and introduced a heuristic method based on nearest-neighbor clustering and routing. This approach exhibited superior execution time when compared to state-of-the-art heuristics. Suwannarongsri [22] addressed time window constraints in task execution and applied the flower pollination algorithm to solve the problem. The results demonstrated superior performance compared to traditional heuristic algorithms, with the added benefit of reduced computational time. Li et al. [23] designed a method based on deep reinforcement learning and solved the constrained routing problem of heterogeneous robots.

The mTSP and mVRP models have improved the expression of the MRTA problem. However, in certain real-world scenarios, the selection of the robot's starting and ending points becomes a relevant consideration. To address this, researchers extended the range of optimization variables by modeling the MRTA problem as a Location-Routing Problem. Lopes et al. [24] introduced a hybrid genetic algorithm to address the Capable Location Routing Problem. The proposed algorithm underwent testing on three sets of instances, and a comparative analysis was conducted with other published methods. The results indicated that, despite its relative simplicity, the method proved to be competitive. Xu et al. [25] took into account the constraints of hard time windows, vehicle capacity, and vehicle return cost within a collaborative logistics network. They introduced a bi-layer programming model to effectively address the Multi-Station Location-Routing Problem under these constraints. Wang et al. [26] proposed a two-stage solution based on deep reinforcement learning for the problem of emergency medical institution selection.

Table 1 provides a summary and comparison of the characteristics of the literature referenced above. Observing the existing literature, it is evident that the predominant focus is on optimizing the total travel distance of robots, with a common emphasis on isomorphic robots. Few studies simultaneously address heterogeneous robots and tasks while optimizing multiple objectives concurrently. Furthermore, each piece of literature establishes the MRTA problem using a fixed mathematical model, and there is a notable absence of work considering the unification of multiple models.

3. System model and problem formulation

3.1. System model

Consider a MRS comprising N_r robots $\mathcal{R} = \{1, 2, \dots, N_r\}$, a task set $\mathcal{T} = \{1, 2, \dots, N_t\}$, and a set consisting of start and end points $\mathcal{G} = \{1, 2, \dots, N_g\}$. In this system, we assume the presence of single-task robots and single-task robot tasks. This implies that each robot can only engage in one task simultaneously, and concurrently, each task is exclusive to a single robot. It is important to note that robots and tasks exhibit heterogeneity, with each robot possessing distinct capabilities, and each task having unique requirements. To capture this diversity, we assign different robots to execute various tasks, each with its own efficiency. Here, e_{ij} represents the efficiency of robot i in performing task j , and v_i represents the moving speed of robot i .

The objective of MRTA is to systematically plan the task execution sequence for each robot to optimize specific indicators within the MRS. In this context, we denote the execution sequence of the robot as $\mathcal{L} = \{L_1, L_2, \dots, L_{N_r}\}$, where L_i signifies the execution sequence of the i th robot. This sequence is comprised of non-repeating integers, and each element denotes the ID of the assigned task. Additionally, I_i and O_i denote the entry and exit points through which the robot i enters and exits the task area. Next, we will introduce the execution time model and transfer time model, respectively.

3.1.1. Execution time model

For each robot, its overall task execution time can be determined according to its task execution sequence, and the specific calculation method is as shown in Eq. (1). The primary objective of this model is not to account for the transfer time of the robot between different task locations but solely to calculate its time consumption during task execution.

$$\mathcal{E}_i = \sum_{j \in L_i} \frac{1}{e_{ij}}, \forall i \in \mathcal{R} \quad (1)$$

3.1.2. Transfer time model

This model primarily delineates the total task transfer time of the robot when the planning information for the robot is determined, and the distribution of task locations is entirely known. For robot i , the total transfer time can be calculated according to Eq. (2):

$$J_i = \sum_{m \in L_i} \sum_{n \in L_i} \frac{1}{v_i} (x_{mn}^i d_{mn} + d_{i1} + d_{iN_r}) \quad (2)$$

where the x_{mn}^i represents whether the tasks with IDs m and n in the task series L_i of robot i are executed consecutively. If the two tasks are executed consecutively, the value is 1, otherwise it is 0. In this context, d_{i1} and d_{iN_r} denote the entrance and exit distances of robot i , respectively. d_{i1} is the distance between the entrance position of robot i and the first task in L_i . Similarly, d_{iN_r} is the distance between the exit position of robot i and the last task in L_i . d_{mn} is the distance of travel path between task m and task n .

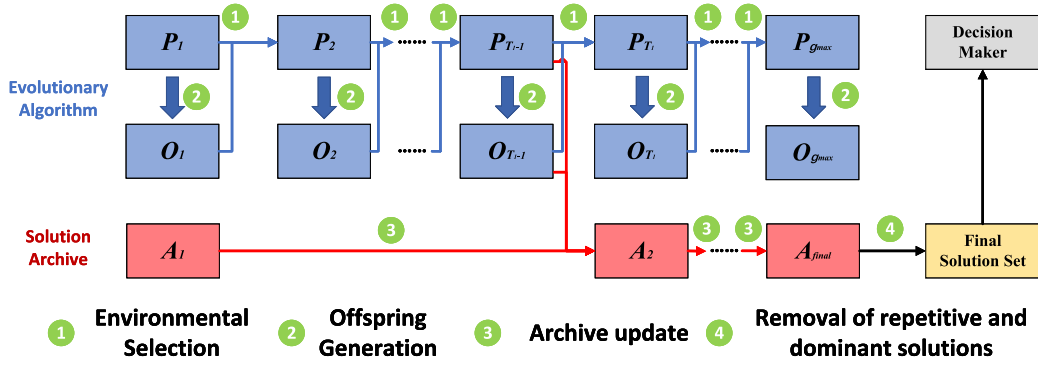


Fig. 2. Illustration of AAUC-IBEA framework.

3.2. Problem formulation

In this paper, we formulate the MRTA problem as a multi-objective optimization problem, incorporating three optimization objectives.

$$\min f_1 = \sum_{i \in R} \mathcal{E}_i \quad (3)$$

$$\min f_2 = \sum_{i \in R} J_i \quad (4)$$

$$\min f_3 = \sum_{i \in R} (\mathcal{E}_i + J_i - \bar{\mathcal{E}} - \bar{J})^2 \quad (5)$$

$$s.t. \quad \sum_{i \in R} |L_i| = N_t \quad (6)$$

$$L_i^j \neq L_i^k, k \in \{1, 2, \dots, |L_i|\}, j \neq k \quad (7)$$

$$L_i \cap L_j = \emptyset, \forall i, j \in R, i \neq j \quad (8)$$

$$L_i^j \in \mathcal{T}, \forall i \in R, \forall j \in \{1, 2, \dots, |L_i|\} \quad (9)$$

$$I_i, O_i \in \mathcal{G}, \forall i \in R \quad (10)$$

Eq. (3) to (5) represent the objective functions. Eq. (3) is used to optimize the total execution time of MRS. Similarly, Eq. (4) is used to optimize the total transfer time of MRS. For each robot, its total time spent is the sum of its execution time and transfer time. Eq. (5) is designed to equalize the total time spent by each robot, effectively optimizing the load balancing within the MRS. Constraint (6) stipulates that the task set must be fully executed, that is, each task is assigned to a robot. Constraint (7) ensures that a single-task robot can only perform one task at the same time. Similar to the previous one, the constraint (8) stipulates that the task is a single-robot task. Constraints (9) and (10) define the effective range of the decision variables and specify the search space of the problem proposed in this article.

The mathematical model presented above constitutes a combinatorial multi-objective optimization problem, posing several challenges in its solution. First, the decision space encoding method involves a non-repeating sequence, akin to the encoding in the TSP. The complexity of the problem's decision space is $O((N_r + N_t)!)^2$. The expansive search space presents a significant challenge to the efficacy of the algorithm's search mechanism. Secondly, the optimization goals of the problem exhibit non-uniform scales. Without effective handling, this can impact the population diversity during the search process. Lastly, addressing the distinct starting and ending points of each robot is crucial. In summary, designing tailored algorithms is imperative to effectively tackle the aforementioned challenges and solve this problem.

4. The proposed approach

4.1. General framework

To effectively address the multi-objective MRTA problem proposed in this paper, we have devised an Indicator-Based Evolutionary Algorithm with Adaptive Archive Update Cycle (AAUC-IBEA). The algorithm's framework is depicted in Fig. 2. Similar to typical evolutionary

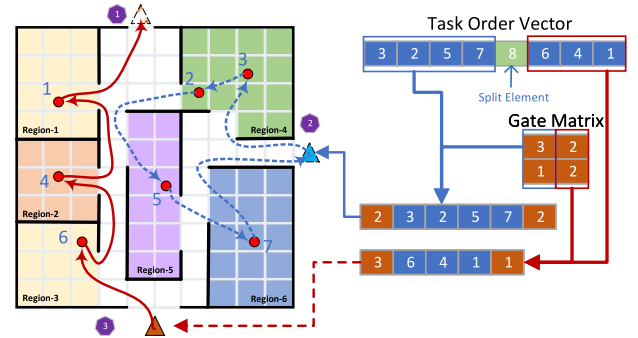


Fig. 3. Illustration of solution representation and decoding process.

algorithms, AAUC-IBEA comprises two main components: offspring generation and environmental selection, operating cyclically in each generation of evolution. However, what sets AAUC-IBEA apart is its utilization of an external archive to store high-quality solutions acquired during the search process. The archive is updated after a specific interval, and noteworthy is the algorithm's dynamic adjustment of the archive update cycle during operation. It should be emphasized that, unlike typical evolutionary algorithms, AAUC-IBEA employs the maximum number of function evaluations as the termination condition for the algorithm. Upon completion, the archive undergoes screening to eliminate duplicate and dominated solutions before being presented as the final population output to the decision-maker.

4.2. Solution encoding

In this paper, we used a combined coding method, which consists of two parts, the task order vector and the gate matrix. The task order vector dictates the order in which the robot should execute tasks. It is a vector with a length of $N_r + N_t - 1$, where each element is a non-repeating integer ranging from 1 to $N_r + N_t - 1$. It is crucial to note that elements with values exceeding N_t represent segmented elements. These segmented elements divide the task order vector into N_r sub-vectors, with each sub-vector corresponding to the execution order of a robot. On the other hand, the gate matrix specifies the entry and exit positions for each robot within the task area. It is an array with two rows and N_r columns, where the elements are integers from 1 to N_g . Each column of the gate matrix represents the starting and ending points for the same robot. The first row denotes the starting point ID, while the second row denotes the ending point ID.

To better illustrate the encoding format of the solution and the decoding process, we have devised a small scenario, as depicted in Fig. 3. In this scenario, there are two robots, seven tasks, and three gates. The tasks and gates in this scene have been assigned IDs and are marked in the picture. The blue vectors and matrices in the figure

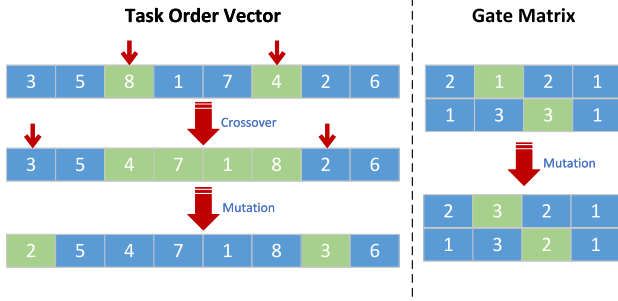


Fig. 4. Illustration of offspring generation process.

represent the original task sequence vectors and gate matrices. For the given solution in this scenario, we identify the split element with value 8 in the task order vector. Subsequently, the task sequence vector is transformed into two sub-vectors with the split element as the boundary. Each sub-vector is then combined with the gate matrix to formulate a task execution sequence. Finally, each robot executes the task according to the assigned execution vector.

Algorithm 1 OffspringGeneration

Require:

Population, P ;

Ensure:

Offspring, O .

- 1: $O \leftarrow \emptyset$;
- 2: **for** every solution p in P **do**
- 3: Apply cross and mutation to solution p following the method outlined in Fig. 4 to obtain offspring o ;
- 4: $O \leftarrow O \cup o$;
- 5: **end for**
- 6: **return** O

4.3. Offspring generation

Given that the solution comprises two parts, distinct operations are devised for each part during the algorithm's offspring generation, as detailed in Fig. 4. For the task order vector, which is sequentially encoded, the 2-opt [27] method is chosen for its crossover operation, while the mutation operation involves randomly exchanging two elements. As for the gate matrix, only the mutation operation is applied, randomly selecting two elements and exchanging them with other values. The pseudocode for offspring generation in the algorithm is presented in Algorithm 1.

Notably, unlike general evolutionary algorithms utilizing tournament selection, the proposed algorithm executes the offspring generation operation for each population. This departure is motivated by two key factors. Firstly, the design of the cross-mutation solution in this article only necessitates a single solution, allowing the generation of a new offspring through operations on a single solution, eliminating the need for tournament selection. Secondly, while increasing the likelihood of generating offspring for highly fit individuals can expedite the convergence of the solution set, it may compromise the algorithm's global search capability. Given that the multi-objective MRTA problem presented in this paper places a premium on solution set diversity, the algorithm opts for the approach of generating offspring for each parent individual to enhance diversity.

4.4. Environmental selection

The environmental selection in AAUC-IBEA closely follows the conventional evolutionary algorithm, as outlined in Algorithm 2. Initially,

the parent and offspring are merged into a set. Subsequently, non-dominated sorting is applied to categorize the solution set into different ranks. The solutions are then integrated into the next generation population in ascending order of rank until the addition of the last rank surpasses the predefined limit, N . To maintain the population size, the hypervolume contribution is used to screen the population in F_l , and the worst one is iteratively discarded. This process continues until the scale of the population decrease to N . It is noteworthy that the T-ENS algorithm [28] is employed for non-dominated sorting in environment selection. The input variable N indicates that the algorithm outputs solutions in all ranks when the solution set size marginally exceeds N . Consequently, in Algorithm 2, the screening of solutions is only in the last rank (F_l).

Algorithm 2 EnvironmentalSelection

Require:

Population, P ;

Offspring, O .

Population Size, N

Ensure:

Next Generation Population, Q .

- 1: $Q \leftarrow P \cup O$;
- 2: $Q = \text{Normalization}(Q)$;
- 3: $[F_1, F_2, \dots, F_l] = \text{NDsort}(Q, N)$;
- 4: $Q = F_1 \cup F_2 \cup \dots \cup F_{l-1}$;
- 5: **while** $|Q| + |F_l| > N$ **do**
- 6: Calculate the hypervolume contribution of each solution in Q ;
- 7: Discard the solution with the minimal hypervolume contribution;
- 8: **end while**
- 9: **return** Q

While environmental selection using hypervolume can enhance selection pressure, accelerating the convergence of the population, it is crucial to acknowledge that exact hypervolume calculations are NP-hard [29]. To enhance the efficiency of the algorithm, we employ the R2HCA [30] for approximate hypervolume contribution calculations. R2HCA, a line-based method, approximates the hypervolume contribution by evaluating the distance from the solution to the hypervolume plane in various directions. The calculation method is represented in Eq. (11).

$$HVC(p, P, \Lambda, r) = \frac{1}{|\Lambda|} \sum_{\lambda \in \Lambda} \min\{\min_{s \in P \setminus \{p\}} \{g^{*2tch}(s|\lambda, p)\}, g^{mch}(r|\lambda, p)\}^m \quad (11)$$

where the Λ is the set of reference vectors and p is the solution in P , r is the reference point of hypervolume. In addition, the quantity $g^{*2tch}(s|\lambda, a)$ and $g^{mch}(r|\lambda, a)$ represents the distance from a to the area enclosed by other solution and the reference point r along the direction vector λ respectively. Their specific calculation methods can be found in Ref. [30].

4.5. Archive update

AAUC-IBEA incorporates an external archive to store high-quality solutions acquired throughout the search process, with the size and update cycle of the archive being dynamically adaptable. Upon each archive update, the interval (T_A) for the subsequent update and the archive size (N_A) are computed using Eqs. (12) and (13), where N is population size. The determination of T_A primarily hinges on two factors: the current search stage (*Stage*) and the non-dominated solution ratio (*Rate*) within the solution set. This design is intentional for two key reasons. Firstly, during the early search stages, offspring solutions have a higher likelihood of dominating the existing population and archives. Frequent updates during this period would lead to computational inefficiency as solutions frequently discarded upon the next update. Thus, a larger update period is maintained in the early search

stage, gradually decreasing as the search progresses. Secondly, to accommodate variations in the convergence speed of the algorithm across different problems, the *Rate* is employed for correction. A low *Rate* suggests ongoing convergence, warranting an increased update period. Conversely, a high *Rate* indicates proximity to a steady state, necessitating a reduced update period. Similarly, the archive size should incrementally grow as the search advances. It should be emphasized that the value of T_A is the number of function evaluations rather than the evolutionary algebra.

$$T_A = \max\{\lfloor 3000 * (1 - Stage + 0.5 * Rate) \rfloor, N\} \quad (12)$$

$$N_A = \lfloor N * (1 + 9 * Stage) \rfloor \quad (13)$$

Algorithm 3 ArchiveUpdate

Require:

Old Archive, A_1 ;
Population, P ;
Offspring, O .
Archive Size, N_A

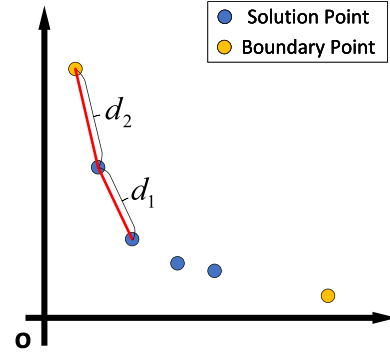
Ensure:

New Archive, A_2 ;
1: $A_2 \leftarrow P \cup O \cup A_1$;
2: $A_2 = \text{NDsort}(A_2, 1)$;
3: **while** $|A_2| > N_A$ **do**
4: Determine the boundary points and assign infinite crowding distance to these boundary points.;
5: Calculate the crowding distance for solutions apart from boundary points.
6: Discard the solution with the smallest crowding distance;
7: **end while**
8: **return** A_2

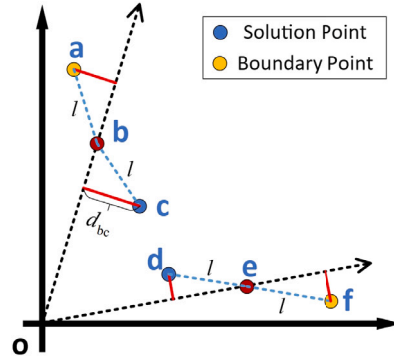
Algorithm 3 outlines the archive update process. Initially, combine the previous archive, parent population, and descendants into a unified collection. Subsequently, conduct non-dominated sorting on the aforementioned set, retaining the non-dominated solutions as the updated archive. Then, assess the size of the archive, and if it does not surpass N_A , return it directly. Conversely, discard solutions one by one based on the modified crowding distance until the archive size equals N_A . It is noteworthy that the archive always preserves the boundary points, which represent the optimal points at each objective value.

The crowding distance method is commonly employed in calculating individual fitness within a population. As depicted in Fig. 5(a), the traditional approach to crowding distance identifies the two points closest to the solution and computes the mean of their Euclidean distances. However, this method exhibits limitations in certain scenarios. For instance, as illustrated in Fig. 5(b), there arises a need to discard a solution between **b** and **e**. However, solutions **b** and **e** possess identical crowding distance l . Consequently, traditional crowding distance becomes inadequate for assessing the quality of these two solutions. In response, we have refined the calculation method for crowding distance. Initially, we locate the two points which have the closest Euclidean distance to the solution concerning the crowding distance in question. Next, determine the vertical distance from these two points to the extension of the solution for the crowding distance. Subsequently, the crowding distance of the solution is obtained by averaging the lengths of the two solid red lines, as shown in Fig. 5(b) for solution **b**. To provide a clearer calculation method, we take the vertical distance d_{bc} between solution $\mathbf{b} = \{b_1, b_2, \dots, b_m\}$ and $\mathbf{c} = \{c_1, c_2, \dots, c_m\}$ as an example for computing the crowding distance. The calculation method is shown in Eq. (14). It should be noted that the solution located ahead of the variables is the one for which the crowding distance needs to be solved, implying that d_{bc} is not equal to d_{cb} .

$$d_{bc} = \left\| \mathbf{c} - \frac{\mathbf{b} \cdot \mathbf{c}}{\|\mathbf{b}\|^2} \cdot \mathbf{b} \right\|$$



(a) General Crowding Distance



(b) Modified Crowding Distance

Fig. 5. Illustration of comparison between general and enhanced crowding distance.

$$= \sqrt{\sum_{i=1}^m \left(c_i - \frac{\sum_{j=1}^m c_j b_j}{\sum_{j=1}^m b_j^2} b_i \right)^2} \quad (14)$$

4.6. Discussion

4.6.1. Challenges and our techniques

Fig. 6 summarizes the challenges addressed in this paper along with the corresponding techniques employed. In response to Challenge 1, we devised a unified mathematical model and a solution encoding method. However, the introduction of an enhanced model led to two additional challenges, namely challenges 2 and 3. To tackle challenge 3, we employed hypervolume as an environmental selection indicator, with the objective of intensifying environmental selection pressure to enhance convergence speed. Addressing challenge 2, we incorporated an external archive into the algorithm to accommodate a larger number of high-quality solutions discovered during the search. Confronting challenges 4 and 5 arising from the establishment of external archives, we implemented an adaptive archive update mechanism and utilized an improved crowding distance metric for truncation, respectively.

4.6.2. Model transformation

This subsection details how to adapt the model proposed in this paper to various mathematical models for addressing MRTA problems. The key challenge lies in configuring the gate matrix. In the context of mTSP, as all robots commence and conclude their tasks at the same location, all elements in the gate matrix must assume same values, with the mutation probability set to 0. Similarly to mVRP necessitates presetting gate matrix parameters for each robot at specified start and end points, with the mutation probability set to 0. In contrast, for LRP, where the robot's starting and ending points serve as planning information, prior constraints on the gate matrix are lifted. In contrast, OAP eliminates the need for a gate matrix and can determine task allocation through task sequence vectors.

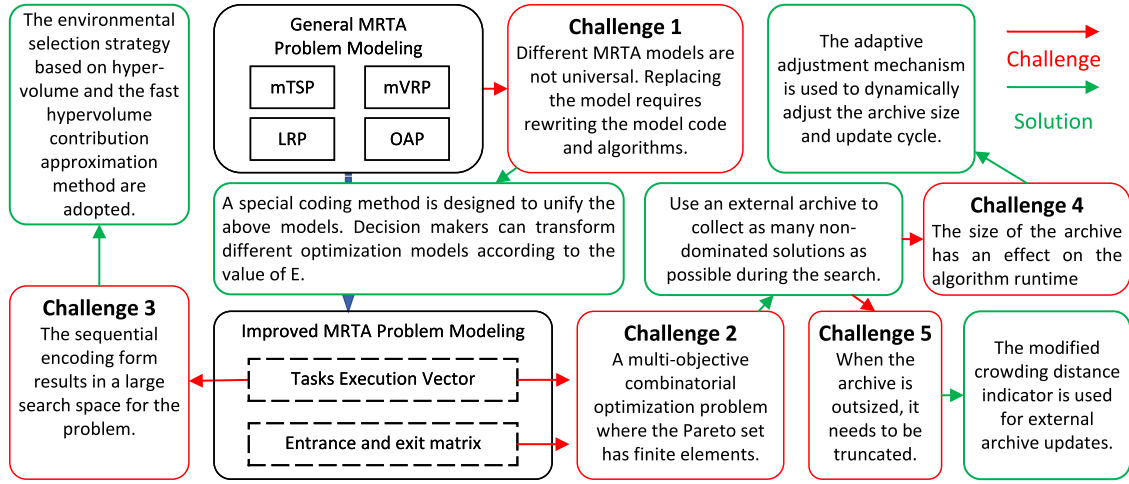


Fig. 6. Challenges and our techniques used in this paper.

4.6.3. Complexity analysis

In the offspring generation phase, each 2-opt operation requires $O(1)$ operations, resulting in a computational complexity of $O(N)$ for generating N offspring. In the environment selection stage, T-ENS has a computational complexity of $O(mN \log N / \log m)$, the standardization of the population target value requires $O(mN)$ calculations, and the hypervolume approximation calculation based on R2 requires $O(N^2|A|)$ calculations. During the archive update phase, finding the non-dominated solution requires $O(mN^2)$ calculations, and calculating an improved crowding distance requires $O(N^2)$ calculations. In the worst case, the above crowding distance needs to be calculated $2N$ times, leading to a computational complexity of $O(N^3)$ for archive update. In summary, considering that A is larger than N , the computational complexity of AAUC-IBEA is $O(N^2|A|)$.

5. Computational experiment

5.1. Comparison algorithm

To assess the effectiveness and efficiency of the proposed algorithm in addressing the MRTA problem, we conducted comparisons with eight state-of-the-art evolutionary multi-objective algorithms. The following provides a brief introduction of comparison algorithms.

MOEA/D-2TH [31] is a variant of MOEA/D, transforming the multi-objective problem into multiple single-objective sub-problems optimized collaboratively. It employs a modified Chebyshev function to measure each sub-problem.

NSGA-II [32] is a Pareto-based multi-objective optimization algorithm that utilizes non-dominated sorting and crowding distance for population screening, demonstrating effectiveness across various test problems.

PREA [33] is a promising-region-based evolutionary many-objective algorithm with the ratio-based indicator. It identifies a promising region in the objective space using the ratio-based indicator with infinite norm, ensuring population diversity with a strategy based on parallel distance.

R2HCA-EMOA [34] is a hypervolume-based evolutionary multi-objective optimization algorithm for many-objective optimization. It approximates hypervolume contribution using an R2 indicator variant.

SPEA2SDE [35] combines SPEA2 with shift-based density estimation (SDE), evaluating convergence and diversity of solution sets in high-dimensional problems and preventing dominance resistance.

TS-NSGA-II [36] is a hybrid algorithm for combinatorial optimization problems, improved in initialization, genetic operators, and constraint processing for more effective solutions to multi-objective combinatorial optimization problems.

Table 2

Instance parameters.

Problem	N_r	N_i	KR	KT	FE_{max}
Instance-1	3	20	[0,0,1,2,0]	[7,3,5,5]	100 000
Instance-2	3	40	[0,1,1,1,0]	[12,4,14,10]	100 000
Instance-3	3	60	[2,0,0,0,1]	[18,13,18,11]	100 000
Instance-4	5	50	[1,1,0,1,2]	[13,12,14,11]	150 000
Instance-5	5	75	[1,1,2,1,0]	[25,16,22,12]	150 000
Instance-6	5	100	[1,1,1,2,0]	[19,30,21,30]	150 000
Instance-7	7	60	[3,1,1,2,0]	[11,21,16,12]	200 000
Instance-8	7	80	[3,0,0,3,1]	[20,24,15,21]	200 000
Instance-9	7	100	[0,0,2,3,2]	[31,20,24,25]	200 000
Instance-10	10	100	[1,3,3,2,1]	[27,20,27,26]	300 000
Instance-11	10	150	[2,3,1,2,2]	[38,25,45,42]	300 000
Instance-12	10	200	[1,1,5,2,1]	[49,66,36,49]	300 000
Instance-13	15	120	[2,5,1,4,3]	[33,30,28,29]	400 000
Instance-14	15	150	[4,3,1,6,1]	[33,44,30,43]	400 000
Instance-15	15	180	[5,1,4,2,3]	[35,51,43,51]	400 000
Instance-16	20	150	[4,4,6,2,4]	[40,31,36,43]	600 000
Instance-17	20	200	[4,4,2,4,6]	[48,48,50,54]	600 000
Instance-18	20	250	[3,5,5,6,1]	[58,73,53,66]	600 000

Table 3

Robot-task execution efficiency.

Robot Type	Task Type			
	Task 1	Task 2	Task 3	Task 4
Robot 1	0.15	0.25	0.13	0.33
Robot 2	0.65	0.01	0.01	0.01
Robot 3	0.01	0.10	0.33	0.10
Robot 4	0.23	0.45	0.01	0.20
Robot 5	0.10	0.15	0.10	0.50

Two-Arch2 [37] is a significantly improved two-archive algorithm with different selection principles (indicator-based and Pareto-based) assigned to the two archives. It features a novel Lp-norm-based diversity maintenance scheme for MaOPs.

VaEA [38] is a vector angle-based evolutionary algorithm for unconstrained many-objective optimization problems. Utilizing the worse-elimination principle, it conditionally replaces worse solutions concerning convergence, maintaining a balance between convergence and diversity.

5.2. Experimental settings

5.2.1. Test instances

In this paper, we establish 18 test instances of varying sizes, with their parameter configurations detailed in Table 2. It is essential to

highlight that the task points in these instances are randomly generated, with the problem type designated as the LRP problem. The rationale behind selecting the LRP problem lies in its possession of the largest search space size among other mathematical models. Consequently, if the proposed algorithm demonstrates superior performance in the LRP problem, it suggests its potential for yielding better outcomes in other, simpler problem models as well. Common parameters in combinatorial optimization problems are employed to denote their distinctions. Here, N_r signifies the number of robots, N_t denotes the number of tasks, K and KT represent the quantities of robots and tasks for different categories respectively, while FE_{max} indicates the maximum number of function evaluations for algorithms. Given the heterogeneity of robots and tasks in this paper, we adopt the execution efficiency, as outlined in Table 3, to characterize the effectiveness of robots when undertaking various task types.

5.2.2. Parameters on algorithms

As various algorithms employ distinct evolutionary strategies, to ensure a fair comparison, all algorithms utilize the number of function evaluations as the termination condition. Regarding algorithm parameters, the number of direction vectors for MOEA/D-2TH is set to 100, the size of Λ for R2HCA-EMOA is set to 100, and for Two-Arch2, CAsize and p are set to 100 and 0.3, respectively. Additionally, the population size for each algorithm is set to 100. It is important to note that the source code for the comparison algorithms in this study is obtained from platEMO [39], an open-source platform based on MATLAB. To allow these algorithms to solve the model established in this paper, the encoding and decoding methods are standardized with AAUC-IBEA.

5.2.3. Platforms

The software platform we use in our experiments is MATLAB R2022b. The hardware platform is a PC equipped with Intel Core i7-12700H CPU@2.70 GHz, 16-GB RAM.

5.3. Performance metric

To assess the performance of AAUC-IBEA and comparison algorithms, we adopt five evaluation metrics that are widely used in the field of multi-objective optimization. The following is an introduction and settings for these indicators.

Hypervolume(HV) is an indicator to evaluate the convergence and diversity of the population [40]. For HV, we use normalization to process the solution set. The normalized solution $\mathbf{x}' = \{x'_1, x'_2, \dots, x'_m\}$ of $\mathbf{x} = \{x_1, x_2, \dots, x_m\}$ can be obtained from Eq. (15), where $\mathbf{z}^* = \{z_1^*, z_2^*, \dots, z_m^*\}$ is the ideal point, and $\mathbf{z}^{nad} = \{z_1^{nad}, z_2^{nad}, \dots, z_m^{nad}\}$ is the nadir point. The specific configurations for $\mathbf{z}^*$ and $\mathbf{z}^{nad}$ for each instance are shown in Table 4. In addition, the reference point is set to [1.1 1.1 1.1].

$$x'_i = \frac{x_i - z_i^*}{z_i^{nad} - z_i^*}, \quad i \in \{1, 2, \dots, m\} \quad (15)$$

Inverted Generational Distance(IGD) is an indicator used to assess the performance of multi-objective optimization algorithms [41]. It is primarily employed to measure the distance between the solution set generated by an algorithm and the PF of a problem. It can be calculated through Eq. (16).

$$IGD(P, P^*) = \frac{\sum_{p^* \in P^*} \min_{p \in P} d(p, p^*)}{|P^*|} \quad (16)$$

where $d(p, p^*)$ represents the Euclidean distance between the point closest to point p in the solution set P . It can be seen that choosing the right reference points is crucial when calculating IGD. In this paper, We ran all algorithms multiple times and used all their non-dominated solutions as reference points.

C-metric [42] is an indicator used to evaluate the convergence of the solution set. The smaller the value, the better the convergence of

Table 4

Hypervolume normalization parameter settings for each instance.

Problem	$\mathbf{z}^*$	$\mathbf{z}^{nad}$
Instance-1	[95.06, 0.001, 22.62]	[177.0, 122.9, 58.29]
Instance-2	[111.8, 0.000, 55.30]	[354.5, 272.7, 136.2]
Instance-3	[273.5, 0.000, 150.8]	[522.7, 404.6, 243.1]
Instance-4	[147.6, 0.000, 69.35]	[431.4, 310.1, 175.7]
Instance-5	[206.6, 0.001, 140.7]	[657.3, 444.2, 298.0]
Instance-6	[300.6, 0.001, 214.1]	[843.2, 601.0, 405.9]
Instance-7	[184.0, 0.013, 59.50]	[497.2, 348.6, 167.8]
Instance-8	[292.1, 0.015, 139.7]	[625.8, 400.9, 294.2]
Instance-9	[467.8, 0.017, 179.1]	[819.3, 502.6, 367.5]
Instance-10	[274.8, 0.041, 133.2]	[867.8, 489.2, 333.1]
Instance-11	[491.1, 0.052, 290.9]	[1264, 706.7, 594.2]
Instance-12	[712.6, 0.050, 431.3]	[1639, 887.2, 793.0]
Instance-13	[337.2, 0.687, 135.9]	[1009, 429.1, 354.6]
Instance-14	[475.7, 0.272, 218.3]	[1228, 535.1, 519.5]
Instance-15	[628.5, 0.405, 288.9]	[1387, 648.8, 583.8]
Instance-16	[426.7, 1.018, 166.1]	[1231, 585.7, 472.2]
Instance-17	[672.6, 0.789, 286.6]	[1666, 652.8, 633.1]
Instance-18	[871.9, 1.540, 421.3]	[2004, 989.9, 958.0]

the solution set. Spacing [43] is an indicator to measure the diversity of solution sets. Its main method is to judge the standard deviation of the distance between solutions. The smaller the value, the better the diversity of solution sets. Spread [44] is an indicator designed to test the extent to which the solution set covers the Pareto front. It is similar to the previous two indicators. The smaller the value, it means that the solution set covers more of the Pareto front.

The Wilcoxon rank sum test, conducted at a significance level of 0.05, is employed to assess whether one algorithm exhibits a statistically significant difference from another. The symbols "+", "-", and "≈" are used to denote that the comparison algorithm is significantly better than, significantly lower than, or statistically similar to AAUC-IBEA.

5.4. Results and discussion

The proposed algorithm and the comparative algorithms were initially assessed based on the HV and IGD values of their respective solution sets across all instances. The outcomes are presented in Tables 5 and 6. Analyzing the HV results reveals that AAUC-IBEA outperforms PREA slightly, achieving the best outcomes in 11 instances, while PREA excels in 5 instances. In comparison, MOEA/D-2TH secures the third spot only in instance-9. Other algorithms exhibit lower HV values than these top three. On the IGD indicator, AAUC-IBEA demonstrates a significant advantage, securing the best results in 17 out of 18 test instances. PREA performs the best in the remaining cases. These differences stem from the HV-based environmental selection of AAUC-IBEA coupled with distance-based archive updates. This approach might result in discarding solutions with substantial indicator contributions during archive updates, thereby reinforcing spatial uniformity in the solution set over indicator value optimality.

Fig. 7 shows the change of the hypervolume of the solution sets obtained by the proposed algorithm and the comparison algorithms on all instances with FE. The analysis reveals that the HV for all algorithms generally rises with an increase in the number of maximum function evaluations. However, TS-NSGA-II exhibits a minor decrease in the later stages. On smaller-scale problems, MOEA/D-2TH and PREA demonstrate rapid convergence. As the problem size grows, AAUC-IBEA starts showcasing its superiority in larger-scale problems. In Instances 13–18, it sustains the fastest convergence speed, indicating that our proposed algorithm is particularly well-suited for addressing large-scale multi-objective MRTA problems.

Similar to Fig. 7, Fig. 8 illustrates the evolution of the IGD value for the algorithm concerning FE. When using IGD as the evaluation criterion, AAUC-IBEA and PREA consistently demonstrate the fastest convergence across nearly all test instances. It is noteworthy that in

Table 5

Statistical results of HV (mean and standard deviation) obtained with AAUC-IBEA and comparison algorithms on 18 instances.

Problem	AAUC-IBEA	MOEA/D-2TH	NSGA-II	PREA	R2HCA-EMOA	SPEA2SDE	TS-NSGA-II	Two-Arch2	VaEA
Instance-1	9.83E-01 (4.35E-03)	9.77E-01 (6.37E-03)	9.80E-01 (3.70E-03)	9.84E-01 (4.89E-03)	9.81E-01 (5.43E-03)	9.79E-01 (4.67E-03)	9.68E-01 (9.37E-03)	9.84E-01 (4.01E-03)	9.78E-01 (5.17E-03)
Instance-2	9.54E-01 (1.28E-02)	9.53E-01 (1.81E-02)	9.39E-01 (1.59E-02)	9.57E-01 (1.07E-02)	9.40E-01 (1.46E-02)	9.35E-01 (1.46E-02)	9.28E-01 (1.35E-02)	9.49E-01 (1.62E-02)	9.28E-01 (1.13E-02)
Instance-3	9.58E-01 (7.25E-03)	9.44E-01 (1.27E-02)	9.44E-01 (1.02E-02)	9.63E-01 (6.99E-03)	9.40E-01 (9.98E-03)	9.36E-01 (8.01E-03)	8.95E-01 (2.89E-02)	9.52E-01 (6.63E-03)	9.30E-01 (9.39E-03)
Instance-4	9.62E-01 (1.58E-02)	9.56E-01 (1.27E-02)	9.22E-01 (1.33E-02)	9.70E-01 (9.93E-03)	9.20E-01 (1.26E-02)	9.13E-01 (1.55E-02)	9.32E-01 (1.75E-02)	9.53E-01 (1.17E-02)	9.16E-01 (1.74E-02)
Instance-5	9.51E-01 (1.60E-02)	9.43E-01 (2.10E-02)	9.06E-01 (2.85E-02)	9.38E-01 (1.09E-02)	9.08E-01 (1.92E-02)	9.04E-01 (1.83E-02)	9.27E-01 (1.68E-02)	9.29E-01 (1.78E-02)	9.00E-01 (2.04E-02)
Instance-6	9.50E-01 (1.30E-02)	9.40E-01 (1.23E-02)	8.92E-01 (1.92E-02)	9.53E-01 (1.19E-02)	8.89E-01 (1.53E-02)	8.72E-01 (1.64E-02)	9.08E-01 (1.77E-02)	9.28E-01 (1.12E-02)	8.75E-01 (1.62E-02)
Instance-7	9.52E-01 (1.42E-02)	9.42E-01 (1.85E-02)	9.15E-01 (2.01E-02)	9.50E-01 (1.44E-02)	9.19E-01 (1.69E-02)	9.13E-01 (1.63E-02)	9.20E-01 (1.55E-02)	9.31E-01 (1.51E-02)	9.11E-01 (1.23E-02)
Instance-8	9.57E-01 (8.76E-03)	9.46E-01 (1.56E-02)	9.18E-01 (1.40E-02)	9.67E-01 (8.56E-03)	9.18E-01 (1.32E-02)	9.07E-01 (1.32E-02)	9.22E-01 (1.24E-02)	9.29E-01 (1.54E-02)	9.05E-01 (1.06E-02)
Instance-9	9.47E-01 (1.00E-02)	9.59E-01 (1.56E-02)	9.02E-01 (1.81E-02)	9.52E-01 (9.64E-03)	9.05E-01 (9.41E-03)	8.90E-01 (1.28E-02)	9.01E-01 (1.76E-02)	9.10E-01 (1.39E-02)	8.91E-01 (1.12E-02)
Instance-10	9.50E-01 (1.27E-02)	9.39E-01 (1.84E-02)	8.81E-01 (2.02E-02)	9.42E-01 (1.63E-02)	8.70E-01 (1.57E-02)	8.58E-01 (1.41E-02)	8.93E-01 (1.81E-02)	9.07E-01 (2.02E-02)	8.59E-01 (1.53E-02)
Instance-11	9.38E-01 (1.97E-02)	9.11E-01 (2.90E-02)	8.27E-01 (1.81E-02)	9.28E-01 (2.23E-02)	8.16E-01 (2.24E-02)	7.99E-01 (1.53E-02)	8.43E-01 (2.05E-02)	8.66E-01 (1.69E-02)	8.11E-01 (1.88E-02)
Instance-12	9.20E-01 (2.09E-02)	8.80E-01 (4.20E-02)	8.07E-01 (2.57E-02)	8.97E-01 (1.85E-02)	7.97E-01 (1.86E-02)	7.82E-01 (2.20E-02)	8.23E-01 (3.87E-02)	8.37E-01 (2.14E-02)	7.86E-01 (1.95E-02)
Instance-13	9.27E-01 (1.67E-02)	9.08E-01 (2.10E-02)	8.40E-01 (1.55E-02)	9.21E-01 (1.45E-02)	8.33E-01 (1.71E-02)	8.19E-01 (1.60E-02)	8.56E-01 (1.71E-02)	8.70E-01 (1.70E-02)	8.23E-01 (1.78E-02)
Instance-14	9.38E-01 (1.32E-02)	9.23E-01 (1.74E-02)	8.59E-01 (1.83E-02)	9.32E-01 (1.46E-02)	8.45E-01 (1.48E-02)	8.35E-01 (1.53E-02)	8.66E-01 (2.25E-02)	8.79E-01 (1.65E-02)	8.32E-01 (1.69E-02)
Instance-15	9.35E-01 (2.10E-02)	9.02E-01 (2.62E-02)	8.39E-01 (1.89E-02)	9.27E-01 (1.84E-02)	8.31E-01 (1.88E-02)	8.14E-01 (2.16E-02)	8.40E-01 (2.46E-02)	8.60E-01 (1.58E-02)	8.10E-01 (1.51E-02)
Instance-16	9.38E-01 (1.09E-02)	9.27E-01 (1.91E-02)	8.49E-01 (1.56E-02)	9.19E-01 (2.02E-02)	8.26E-01 (1.59E-02)	8.29E-01 (1.62E-02)	8.61E-01 (1.95E-02)	8.81E-01 (1.72E-02)	8.23E-01 (1.81E-02)
Instance-17	9.49E-01 (1.65E-02)	8.67E-01 (3.35E-02)	8.06E-01 (2.28E-02)	9.17E-01 (2.37E-02)	8.02E-01 (1.91E-02)	7.88E-01 (1.87E-02)	8.19E-01 (2.11E-02)	8.48E-01 (2.14E-02)	7.79E-01 (1.95E-02)
Instance-18	9.52E-01 (1.46E-02)	8.98E-01 (3.26E-02)	8.19E-01 (2.31E-02)	9.26E-01 (1.87E-02)	8.04E-01 (1.92E-02)	7.96E-01 (1.65E-02)	8.25E-01 (2.50E-02)	8.41E-01 (1.88E-02)	7.85E-01 (1.54E-02)
+ / - / $\approx$	-	0/13/5	0/17/1	2/8/8	0/17/1	0/18/0	0/18/0	0/16/2	0/18/0

Table 6

Statistical results of IGD (mean and standard deviation) obtained with AAUC-IBEA and comparison algorithms on 18 instances.

Problem	AAUC-IBEA	MOEA/D-2TH	NSGA-II	PREA	R2HCA-EMOA	SPEA2SDE	TS-NSGA-II	Two-Arch2	VaEA
Instance-1	2.31E-02 (5.11E-03)	5.59E-02 (6.59E-03)	7.30E-02 (1.56E-02)	3.00E-02 (3.85E-03)	4.30E-02 (8.01E-03)	4.31E-02 (5.81E-03)	7.79E-02 (1.68E-02)	3.37E-02 (3.27E-03)	3.59E-02 (4.76E-03)
Instance-2	4.63E-02 (1.43E-02)	9.56E-02 (1.42E-02)	1.40E-01 (3.43E-02)	5.17E-02 (1.04E-02)	7.85E-02 (1.40E-02)	8.66E-02 (2.33E-02)	1.37E-01 (2.52E-02)	6.58E-02 (2.24E-02)	9.16E-02 (1.38E-02)
Instance-3	3.54E-02 (5.03E-03)	9.06E-02 (1.32E-02)	1.05E-01 (1.29E-02)	4.16E-02 (5.30E-03)	6.45E-02 (1.17E-02)	6.90E-02 (9.21E-03)	1.81E-01 (5.27E-02)	4.93E-02 (6.88E-02)	8.50E-02 (9.65E-03)
Instance-4	9.05E-02 (2.63E-02)	1.45E-01 (2.98E-02)	2.64E-01 (6.66E-02)	8.42E-02 (1.79E-02)	1.73E-01 (3.23E-02)	1.86E-01 (3.49E-02)	1.72E-01 (5.24E-02)	1.10E-01 (2.50E-02)	1.69E-01 (4.01E-02)
Instance-5	1.29E-01 (5.50E-02)	1.87E-01 (7.66E-02)	3.72E-01 (1.65E-01)	1.76E-01 (4.57E-02)	2.87E-01 (9.71E-02)	3.01E-01 (8.35E-02)	2.14E-01 (7.28E-02)	2.05E-01 (6.11E-02)	3.06E-01 (9.20E-02)
Instance-6	4.92E-02 (1.52E-02)	9.95E-02 (1.96E-02)	1.76E-01 (4.50E-02)	5.16E-02 (1.63E-02)	1.46E-01 (2.59E-02)	1.73E-01 (2.97E-02)	1.21E-01 (3.04E-02)	7.60E-02 (1.58E-02)	1.60E-01 (2.89E-02)
Instance-7	7.70E-02 (3.01E-02)	1.50E-01 (5.24E-02)	2.51E-01 (6.82E-02)	9.49E-02 (4.11E-02)	1.38E-01 (4.46E-02)	1.50E-01 (4.30E-02)	1.64E-01 (4.60E-02)	1.22E-01 (4.08E-02)	1.46E-01 (3.53E-02)
Instance-8	9.79E-02 (2.59E-02)	1.77E-01 (4.23E-02)	3.24E-01 (6.60E-02)	8.06E-02 (1.85E-02)	1.93E-01 (3.75E-02)	2.25E-01 (4.87E-02)	2.20E-01 (4.59E-02)	1.51E-01 (4.00E-02)	2.15E-01 (3.63E-02)
Instance-9	8.23E-02 (1.28E-02)	1.10E-01 (2.84E-02)	2.11E-01 (4.60E-02)	7.69E-02 (1.48E-02)	1.37E-01 (1.81E-02)	1.54E-01 (2.28E-02)	1.29E-01 (2.72E-02)	1.18E-01 (1.72E-02)	1.43E-01 (2.07E-02)
Instance-10	7.10E-02 (2.99E-02)	1.74E-01 (6.81E-02)	2.37E-01 (5.33E-02)	8.67E-02 (2.92E-02)	1.68E-01 (3.40E-02)	1.86E-01 (3.26E-02)	1.63E-01 (5.29E-02)	1.29E-01 (4.00E-02)	1.73E-01 (3.01E-02)

(continued on next page)

Table 6 (continued).

Problem	AAUC-IBEA	MOEA/D-2TH	NSGA-II	PREA	R2HCA-EMOA	SPEA2SDE	TS-NSGA-II	Two-Arch2	VaEA
Instance-11	8.31E-02 (1.92E-02)	1.80E-01 (5.34E-02)	3.01E-01 (3.23E-02)	9.70E-02 (2.67E-02)	2.45E-01 (3.12E-02)	2.73E-01 (3.40E-02)	1.95E-01 (3.05E-02)	1.79E-01 (2.75E-02)	2.36E-01 (3.40E-02)
Instance-12	1.01E-01 (4.08E-02)	2.92E-01 (1.07E-01)	4.84E-01 (8.84E-02)	1.58E-01 (6.10E-02)	3.40E-01 (6.50E-02)	3.86E-01 (5.32E-02)	2.98E-01 (8.10E-02)	2.89E-01 (7.92E-02)	3.36E-01 (6.25E-02)
Instance-13	8.84E-02 (2.06E-02)	2.30E-01 (5.62E-02)	3.48E-01 (5.03E-02)	9.34E-02 (2.82E-02)	2.24E-01 (3.56E-02)	2.55E-01 (3.23E-02)	2.10E-01 (4.46E-02)	1.69E-01 (3.33E-02)	2.23E-01 (3.46E-02)
Instance-14	8.68E-02 (2.27E-02)	1.95E-01 (4.66E-02)	3.33E-01 (4.67E-02)	1.07E-01 (3.09E-02)	2.32E-01 (2.88E-02)	2.61E-01 (3.37E-02)	2.33E-01 (5.54E-02)	1.88E-01 (3.21E-02)	2.47E-01 (4.66E-02)
Instance-15	1.16E-01 (3.23E-02)	2.13E-01 (4.39E-02)	3.64E-01 (5.15E-02)	1.22E-01 (3.20E-02)	2.70E-01 (3.90E-02)	3.07E-01 (3.72E-02)	2.82E-01 (4.87E-02)	2.20E-01 (3.47E-02)	2.84E-01 (3.75E-02)
Instance-16	8.55E-02 (1.82E-02)	2.07E-01 (5.35E-02)	3.57E-01 (5.48E-02)	1.19E-01 (4.38E-02)	2.88E-01 (4.79E-02)	2.78E-01 (4.04E-02)	2.49E-01 (4.66E-02)	1.94E-01 (3.68E-02)	2.57E-01 (3.89E-02)
Instance-17	1.06E-01 (3.30E-02)	3.56E-01 (8.80E-02)	4.80E-01 (5.54E-02)	1.56E-01 (4.62E-02)	3.30E-01 (4.89E-02)	3.73E-01 (4.52E-02)	3.19E-01 (4.96E-02)	2.67E-01 (4.61E-02)	3.42E-01 (4.98E-02)
Instance-18	1.05E-01 (3.09E-02)	3.27E-01 (1.09E-01)	4.92E-01 (7.35E-02)	1.51E-01 (3.90E-02)	3.49E-01 (4.02E-02)	3.57E-01 (4.41E-02)	3.31E-01 (7.98E-02)	2.89E-01 (5.39E-02)	3.41E-01 (4.56E-02)
+ / - / ≈	-	0/18/0	0/18/0	3/13/2	0/18/0	0/18/0	0/18/0	0/18/0	0/18/0

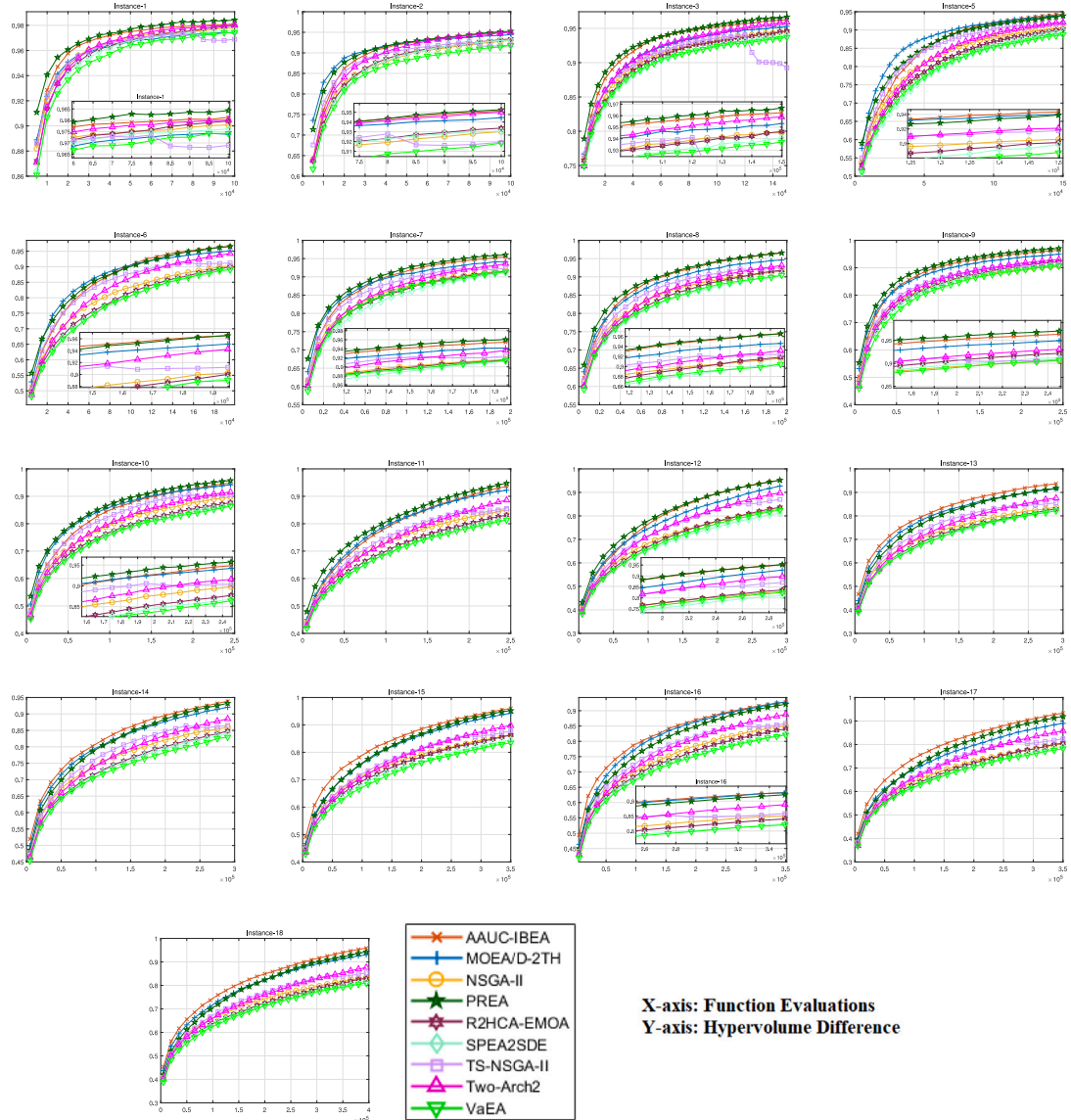


Fig. 7. Convergence analysis with mean HV of 30 times run of all algorithms for each instance.

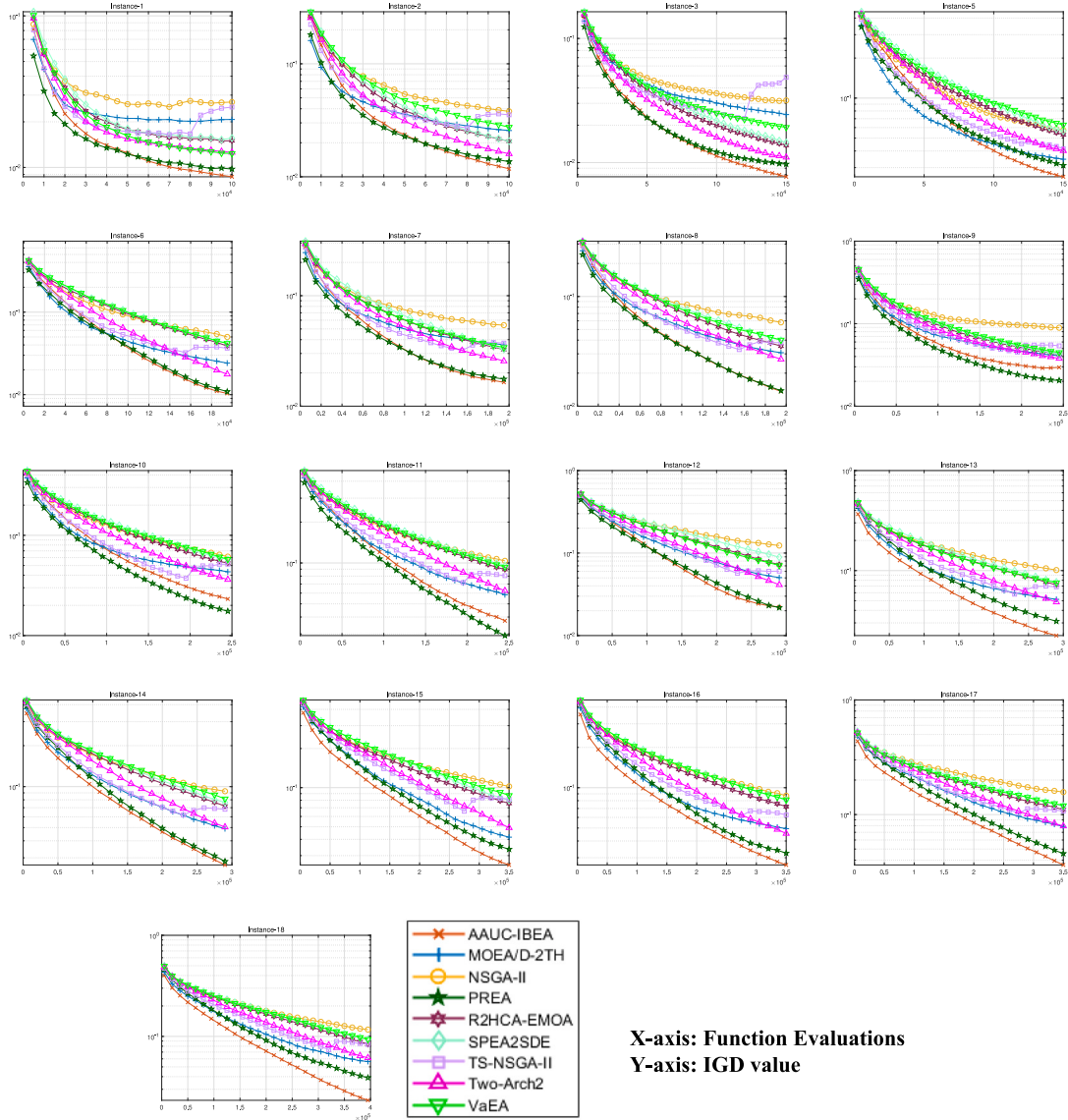


Fig. 8. Convergence analysis with mean IGD of 30 times run of all algorithms for each instance.

the later stages of the AAUC-IBEA search, the archived IGD indicator consistently exhibits a declining trend. This is attributed to the fact that, in the advanced stages of the algorithm, the archive size can almost accommodate all non-dominated solutions obtained by all algorithms. Consequently, there is no need for the algorithm to perform distance-based truncation operations, allowing the archive to store a maximal number of non-dominated solutions and thereby further reducing the IGD value of the archive.

To assess the algorithm's solving efficiency, we initially examined its runtime performance, as depicted in Fig. 9. The results align with our expectations, categorizing the running time of all algorithms into two distinct groups. The three algorithms, AAUC-IBEA, R2HCA-EMOA, and MOEA/D-2TH, exhibit lengthier runtimes, whereas the remaining algorithms consume comparatively less time. Both the proposed algorithm and R2HCA-EMOA are selected based on the HV environment, contributing to their increased time consumption. In contrast, MOEA/D-2TH employs a global replacement strategy, elevating its computational complexity. Other algorithms predominantly adopt Pareto strategies or utilize low-complexity indicators, resulting in faster running times. Subsequently, we examined the composition of the solution sets obtained by each algorithm. The two sub-figures in Fig. 10 display the number of non-dominated solutions in the solution set and

the proportion of non-dominated solutions, with all duplicate solutions removed. Notably, aside from Two-Arch2, TS-NSGA-II, and MOEA/D-2TH, the solution sets acquired by other algorithms exclusively consist of non-dominated solutions. The proposed algorithm not only yields a substantial number of solution sets but also ensures that all solutions in the set are non-dominated, underscoring the efficacy of AAUC-IBEA in solution generation.

Fig. 11 illustrates the C-Metric numerical comparison between the proposed algorithm and the comparison algorithms, where lower values indicate superior solution set convergence. Notably, AAUC-IBEA, MOEA/D-2TH, and PREA consistently achieved lower C-Metric values across all instances. On small-scale instances, MOEA/D-2TH demonstrated the best convergence. In contrast, on large-scale instances, the solution set obtained by AAUC-IBEA exhibited the most favorable convergence. In Fig. 12, the numerical comparison of Spacing and Spread indicators further reveals that the solution set generated by AAUC-IBEA displays commendable uniform distribution and Pareto front coverage.

Finally, to observe the distribution of solutions, we depict the solution set distribution obtained by all algorithms on Instance-1, as illustrated in Fig. 13. Each subgraph represents the solution set distribution of an algorithm, with red points representing the solutions obtained by the algorithm, and green points indicating the Pareto set of

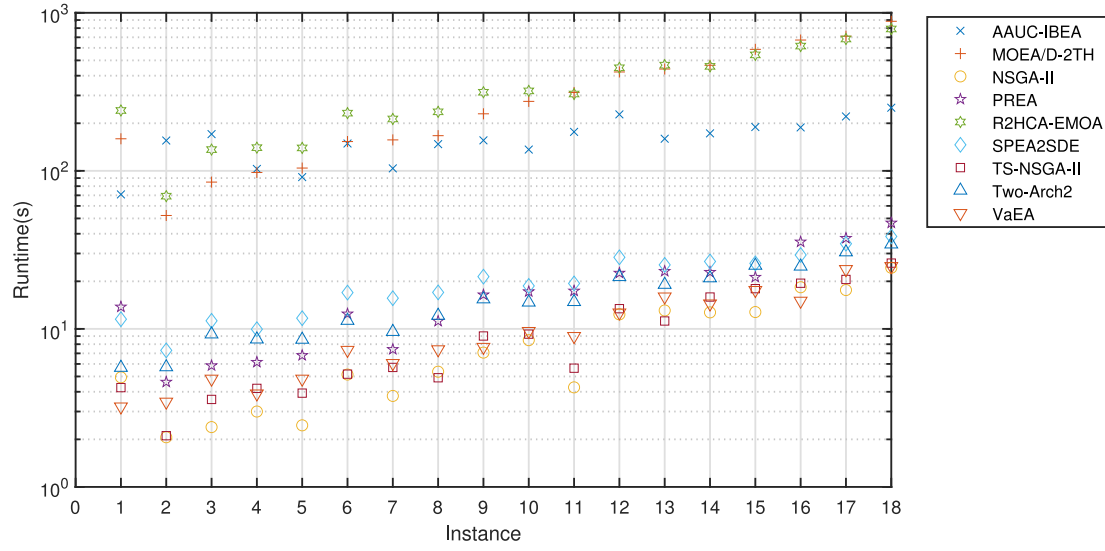


Fig. 9. Comparison of runtime between the proposed algorithm and the comparison algorithms on 18 instances.

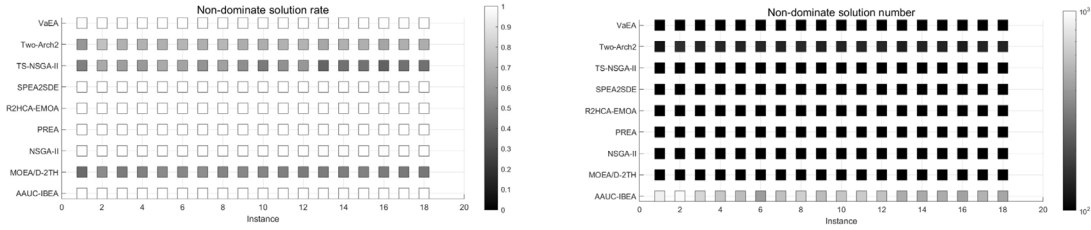


Fig. 10. Comparison of non-dominant solutions number and rate between the proposed algorithm and the comparison algorithm on 18 instances.

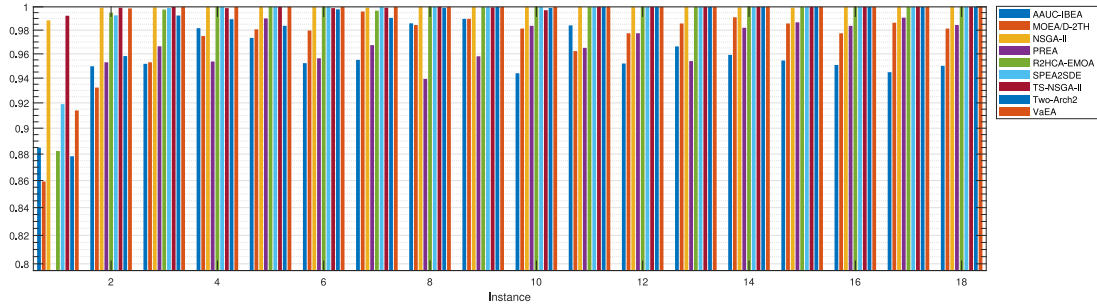


Fig. 11. Comparison of C-Metric value between the proposed algorithm and the comparison algorithm on 18 examples.

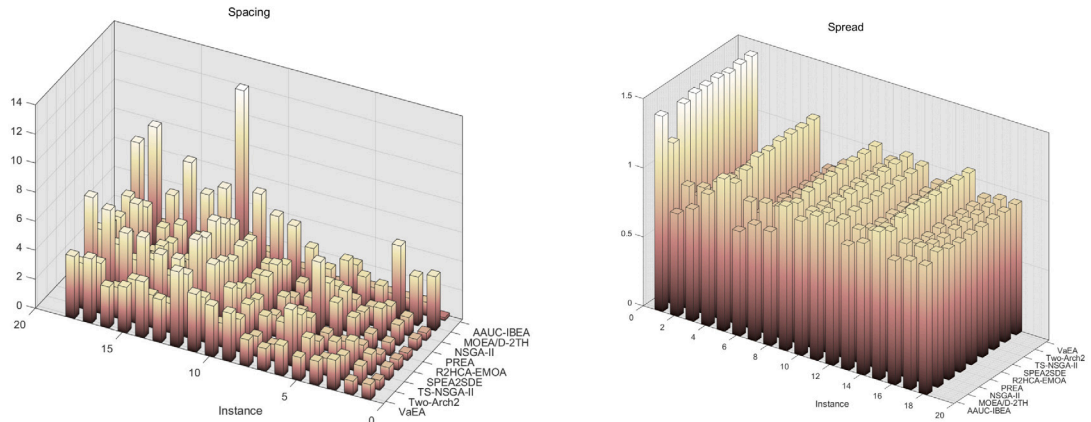


Fig. 12. Comparison of Spacing and Spread value between the proposed algorithm and the comparison algorithm on 18 examples.

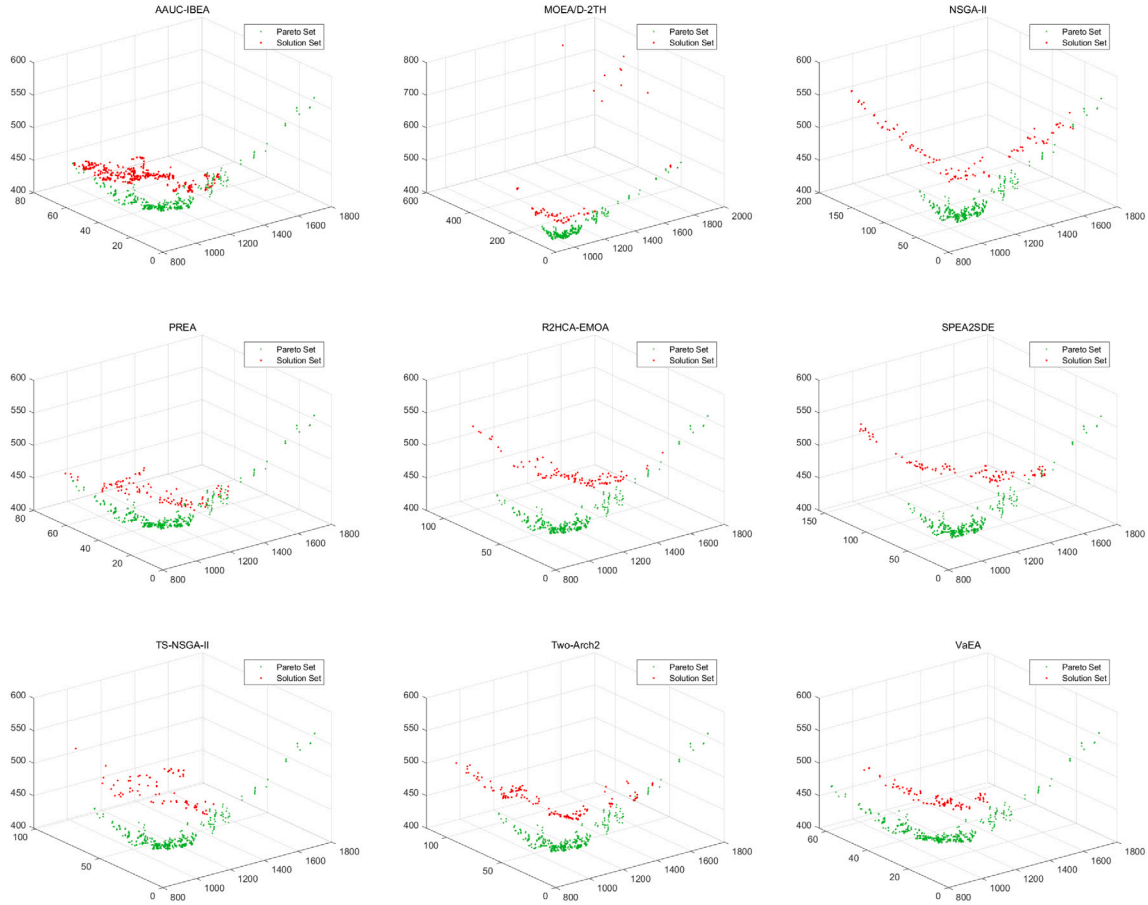


Fig. 13. The solution distribution of AAUC-IBEA and comparison algorithms on Instance-1.

Table 7

Statistical results of HV and IGD obtained with AAUC-IBEA and comparison algorithms on 4 real-world problems.

Problem	Indicator	Bayg29	Berlin52	Bier127	Kro200
AAUC-IBEA	HV	9.30E-01(1.01E-02)	8.05E-01(1.07E-02)	5.58E-01(3.56E-02)	6.89E-01(1.56E-02)
MOEA/D-2TH	HV	9.26E-01(2.00E-02)≈	7.54E-01(1.10E-02)–	4.26E-01(3.19E-02)–	5.79E-01(4.92E-02)–
NSGA-II	HV	9.09E-01(2.81E-02)–	7.52E-01(2.05E-02)–	3.41E-01(3.96E-02)–	3.59E-01(3.25E-02)–
PREA	HV	9.53E-01(4.39E-03)+	8.15E-01(1.04E-02)≈	4.53E-01(5.11E-02)–	6.46E-01(3.76E-02)–
R2HCA-EMOA	HV	9.17E-01(1.32E-02)–	7.57E-01(4.01E-02)–	4.39E-01(3.55E-02)–	4.31E-01(1.79E-02)–
SPEA2SDE	HV	8.98E-01(1.72E-02)–	7.38E-01(2.64E-02)–	3.91E-01(5.31E-02)–	4.20E-01(2.82E-02)–
TS-NSGA-II	HV	9.15E-01(3.95E-02)–	7.27E-01(3.86E-02)–	4.20E-01(5.44E-02)–	5.09E-01(3.11E-02)–
Two-Arch2	HV	9.23E-01(1.92E-02)≈	7.88E-01(2.32E-02)≈	5.76E-01(5.81E-02)+	5.43E-01(1.95E-02)–
VaEA	HV	9.00E-01(1.08E-02)–	7.24E-01(3.11E-02)–	3.74E-01(1.28E-02)–	4.02E-01(1.86E-02)–
AAUC-IBEA	IGD	1.63E-00(3.45E-01)	5.13E-00(9.25E-01)	4.29E+01(2.72E-00)	1.16E+01(1.35E-00)
MOEA/D-2TH	IGD	3.50E-00(3.31E-01)–	5.80E-00(1.16E-01)–	4.24E+01(3.16E-00)≈	1.22E+02(3.50E+01)–
NSGA-II	IGD	3.03E-00(1.12E-00)–	8.07E-00(2.66E-00)–	4.79E+01(4.72E-00)–	6.03E+01(1.75E+01)–
PREA	IGD	3.42E-00(1.14E-00)–	5.42E-00(7.93E-01)–	4.72E+01(3.84E-00)–	2.18E+02(2.76E+01)–
R2HCA-EMOA	IGD	3.13E-00(1.08E-00)–	7.52E-00(3.08E-00)–	4.59E+01(5.28E-00)–	1.43E+02(9.64E-00)–
SPEA2SDE	IGD	4.22E-00(1.36E-00)–	7.80E-00(2.36E-00)–	5.23E+01(4.31E-00)–	1.70E+02(2.42E+01)–
TS-NSGA-II	IGD	4.40E-00(1.13E-00)–	1.10E+01(1.73E-00)–	5.69E+01(7.22E-00)–	2.03E+02(5.62E+00)–
Two-Arch2	IGD	3.68E-00(1.52E-00)–	5.16E-00(1.08E-00)–	4.31E+01(2.18E-00)≈	1.86E+02(1.62E+01)–
VaEA	IGD	3.29E-00(5.35E-01)–	9.88E-00(1.98E-00)–	5.94E+01(3.68E-00)–	1.75E+02(3.84E+01)–

Instance-1. The solution sets of NSGA-II, SPEA2SDE, and R2HCA-EMOA exhibit considerable distance from the PF. Although MOEA/D-2TH covers PF effectively, it maintains some distance and includes dominant solutions. VaEA, Two-Arch2, and TS-NSGA-II have converged closer to PF while maintaining relatively good class diversity. The solution sets of AAUC-IBEA and PREA have converged to PF at the left and right ends. Additionally, it is evident that the solution set of AAUC-IBEA is denser than that of PREA.

To further demonstrate the efficacy of the proposed algorithm in tackling real-world challenges, we have included four problems from the TSPLIB [45] benchmark library: Bayg29, Berlin52, Bier127, and

Kro200. These problems are based on real city point data. It is worth noting that the mission points and robot types are randomly generated. We set the maximum number of evaluations for all algorithms on these four problems to 50 000, 100 000, 200 000, and 300 000, respectively, with the number of robots being 3, 5, 10, and 15, respectively. Each algorithm was executed independently 10 times. The results were evaluated solely using the HV and IGD metrics and are summarized in Table 7. The experimental results demonstrate similarities to those obtained with randomly generated instance points. PREA achieved the best results in terms of the HV indicator, while AAUC-IBEA and Two-Arch2 also performed well. Regarding the IGD indicator, AAUC-IBEA

demonstrated significant advantages. These findings underscore the competitiveness of the proposed algorithm in addressing real-world problems.

This section assesses the efficacy of the proposed algorithm and 8 existing advanced algorithms across 18 randomly generated instances of varying sizes and 4 benchmark test problems. Across 5 distinct evaluation indicators, the proposed algorithm consistently yielded favorable results. This further substantiates the effectiveness of the proposed algorithm in addressing the MRTA model outlined in this article.

6. Conclusion

In this paper, we addressed the adaptability of the Multi-Robot Task Allocation (MRTA) model by establishing a unified mathematical formulation. To effectively tackle this model, we devised an evolutionary algorithm incorporating external archiving. The algorithm leverages hypervolume for enhanced environmental selection pressure, expediting convergence, and integrates an adaptive archive update mechanism to improve solution efficiency. Experimental comparisons with state-of-the-art algorithms across different size problems demonstrate the superiority of our method. Future work will focus on two key aspects:

- (1). Constraint handling: Real-world multi-robot systems contend with various constraints, such as robot load and energy considerations, task time windows, logical constraints, and environmental accessibility. Our next steps involve incorporating these constraints into the established model.
- (2). Dynamic task environments: Many task environments exhibit dynamic characteristics, with changes in task type and quantity during execution. Addressing the dynamic nature of tasks will be a crucial and meaningful aspect of our future research.

CRedit authorship contribution statement

Chengxin Wen: Writing – original draft, Software, Methodology, Conceptualization. **Hongbin Ma:** Validation, Supervision, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data will be made available on request

Data availability

Data will be made available on request.

Acknowledgments

This work was partially funded by the National Key Research and Development Plan of China (No. 2018AAA01-01000) and the National Natural Science Foundation of China under grant 62076028.

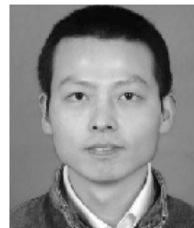
References

- [1] G.P. Das, T.M. McGinnity, S.A. Coleman, L. Behera, A distributed task allocation algorithm for a multi-robot system in healthcare facilities, *J. Intell. Robot. Syst.* 80 (2015) 33–58.
- [2] S. Zhao, B.M. Chen, T.H. Lee, Optimal sensor placement for target localisation and tracking in 2D and 3D, *Internat. J. Control* 86 (10) (2013) 1687–1704.
- [3] M. Khani, A. Ahmadi, H. Hajary, Distributed task allocation in multi-agent environments using cellular learning automata, *Soft Comput.* 23 (2019) 1199–1218.
- [4] C. Liu, A. Kroll, Memetic algorithms for optimal task allocation in multi-robot systems for inspection problems with cooperative tasks, *Soft Comput.* 19 (2015) 567–584.
- [5] L.C. Pimenta, M. Schwager, Q. Lindsey, V. Kumar, D. Rus, R.C. Mesquita, G.A. Pereira, Simultaneous coverage and tracking (SCAT) of moving targets with robot networks, in: *Algorithmic Foundation of Robotics VIII: Selected Contributions of the Eight International Workshop on the Algorithmic Foundations of Robotics*, Springer, 2010, pp. 85–99.
- [6] X. Dai, L. Jiang, Y. Zhao, Cooperative exploration based on supervisory control of multi-robot systems, *Appl. Intell.* 45 (2016) 18–29.
- [7] A. Khamis, A. Hussein, A. Elmogy, Multi-robot task allocation: A review of the state-of-the-art, in: *Cooperative Robots and Sensor Networks 2015*, Springer, 2015, pp. 31–51.
- [8] F. Tang, L.E. Parker, A complete methodology for generating multi-robot task solutions using asymptotic and market-based task allocation, in: *Proceedings 2007 IEEE International Conference on Robotics and Automation*, IEEE, 2007, pp. 3351–3358.
- [9] K. Lerman, C. Jones, A. Galstyan, M.J. Mataric, Analysis of dynamic task allocation in multi-robot systems, *Int. J. Robotics Res.* 25 (3) (2006) 225–241.
- [10] H.W. Kuhn, The Hungarian method for the assignment problem, *Nav. Res. Logist. Q.* 2 (1–2) (1955) 83–97.
- [11] C. Nam, D.A. Shell, Assignment algorithms for modeling resource contention in multirobot task allocation, *IEEE Trans. Autom. Sci. Eng.* 12 (3) (2015) 889–900.
- [12] C. Nam, D.A. Shell, Analyzing the sensitivity of the optimal assignment in probabilistic multi-robot task allocation, *IEEE Robot. Autom. Lett.* 2 (1) (2016) 193–200.
- [13] L. Liu, D.A. Shell, Large-scale multi-robot task allocation via dynamic partitioning and distribution, *Auton. Robots* 33 (2012) 291–307.
- [14] R. Zhang, Q. Lv, J. Li, J. Bao, T. Liu, S. Liu, A reinforcement learning method for human-robot collaboration in assembly tasks, *Robot. Comput.-Integr. Manuf.* 73 (2022) 102227.
- [15] C. Liu, A. Kroll, A centralized multi-robot task allocation for industrial plant inspection by using a* and genetic algorithms, in: *Artificial Intelligence and Soft Computing: 11th International Conference, ICAISC 2012, Zakopane, Poland, April 29–May 3, 2012, Proceedings, Part II 11*, Springer, 2012, pp. 466–474.
- [16] Y. Msala, O. Hamed, M. Talea, M. Aboulfatah, A new method for improving the fairness of multi-robot task allocation by balancing the distribution of tasks, *J. Robotics Control (JRC)* 4 (6) (2023) 743–753.
- [17] C. Wei, Z. Ji, B. Cai, Particle swarm optimization for cooperative multi-robot task allocation: A multi-objective approach, *IEEE Robot. Autom. Lett.* 5 (2) (2020) 2530–2537.
- [18] A. Bogoybayeva, T. Yoon, H. Ko, S. Lim, H. Yun, C. Kwon, A deep reinforcement learning approach for solving the traveling salesman problem with drone, *Transp. Res. C* 148 (2023) 103981.
- [19] R. Zhang, C. Zhang, Z. Cao, W. Song, P.S. Tan, J. Zhang, B. Wen, J. Dauwels, Learning to solve multiple-TSP with time window and rejections via deep reinforcement learning, *IEEE Trans. Intell. Transp. Syst.* 24 (1) (2022) 1325–1336.
- [20] K. Padmanabhan Panchu, M. Rajmohan, R. Sundar, R. Baskaran, Multi-objective optimisation of multi-robot task allocation with precedence constraints, *Def. Sci. J.* 68 (2) (2018).
- [21] C. Sarkar, H.S. Paul, A. Pal, A scalable multi-robot task allocation algorithm, in: *2018 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2018*, pp. 5022–5027.
- [22] S. Suwannarongsri, Solving multiple vehicle routing problems with time constraints by flower pollination algorithm optimization, *WSEAS Trans. Syst.* 19 (2020) 178–187.
- [23] J. Li, Y. Ma, R. Gao, Z. Cao, A. Lim, W. Song, J. Zhang, Deep reinforcement learning for solving the heterogeneous capacitated vehicle routing problem, *IEEE Trans. Cybern.* 52 (12) (2021) 13572–13585.
- [24] R.B. Lopes, C. Ferreira, B.S. Santos, A simple and effective evolutionary algorithm for the capacitated location-routing problem, *Comput. Oper. Res.* 70 (2016) 155–162.
- [25] X. Xu, Y. Zheng, L. Yu, A bi-level optimization model of LRP in collaborative logistics network considered backhaul no-load cost, *Soft Comput.* 22 (2018) 5385–5393.
- [26] S. Wang, J. Zhou, H. Liang, Z. Wang, C. Su, X. Li, A new approach for solving location routing problems with deep reinforcement learning of emergency medical facility, in: *Proceedings of the 8th ACM SIGSPATIAL International Workshop on Security Response using GIS, 2023*, pp. 50–53.
- [27] M. Englert, H. Röglin, B. Vöcking, Worst case and probabilistic analysis of the 2-opt algorithm for the TSP, *Algorithmica* 68 (1) (2014) 190–264.
- [28] X. Zhang, Y. Tian, R. Cheng, Y. Jin, Empirical analysis of a tree-based efficient non-dominated sorting approach for many-objective optimization, in: *2016 IEEE Symposium Series on Computational Intelligence, SSCI, 2016*, pp. 1–8, <http://dx.doi.org/10.1109/SSCI.2016.7850210>.
- [29] K. Bringmann, T. Friedrich, Approximating the volume of unions and intersections of high-dimensional geometric objects, *Comput. Geom.* 43 (6–7) (2010) 601–610.
- [30] K. Shang, H. Ishibuchi, X. Ni, R2-based hypervolume contribution approximation, *IEEE Trans. Evol. Comput.* 24 (1) (2019) 185–192.
- [31] X. Ma, Q. Zhang, G. Tian, J. Yang, Z. Zhu, On tchebycheff decomposition approaches for multiobjective evolutionary optimization, *IEEE Trans. Evol. Comput.* 22 (2) (2018) 226–244, <http://dx.doi.org/10.1109/TEVC.2017.2704118>.

- [32] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE Trans. Evol. Comput.* 6 (2) (2002) 182–197.
- [33] J. Yuan, H.-L. Liu, F. Gu, Q. Zhang, Z. He, Investigating the properties of indicators and an evolutionary many-objective algorithm using promising regions, *IEEE Trans. Evol. Comput.* 25 (1) (2020) 75–86.
- [34] K. Shang, H. Ishibuchi, A new hypervolume-based evolutionary algorithm for many-objective optimization, *IEEE Trans. Evol. Comput.* 24 (5) (2020) 839–852.
- [35] M. Li, S. Yang, X. Liu, Shift-based density estimation for Pareto-based algorithms in many-objective optimization, *IEEE Trans. Evol. Comput.* 18 (3) (2013) 348–365.
- [36] S. Verma, M. Pant, V. Snasel, A comprehensive review on NSGA-II for multi-objective combinatorial optimization problems, *Ieee Access* 9 (2021) 57757–57791.
- [37] H. Wang, L. Jiao, X. Yao, Two_Arch2: An improved two-archive algorithm for many-objective optimization, *IEEE Trans. Evol. Comput.* 19 (4) (2014) 524–541.
- [38] Y. Xiang, Y. Zhou, M. Li, Z. Chen, A vector angle-based evolutionary algorithm for unconstrained many-objective optimization, *IEEE Trans. Evol. Comput.* 21 (1) (2016) 131–152.
- [39] Y. Tian, R. Cheng, X. Zhang, Y. Jin, PlatEMO: A MATLAB platform for evolutionary multi-objective optimization [educational forum], *IEEE Comput. Intell. Mag.* 12 (4) (2017) 73–87.
- [40] E. Zitzler, D. Brockhoff, L. Thiele, The hypervolume indicator revisited: On the design of Pareto-compliant indicators via weighted integration, in: *Evolutionary Multi-Criterion Optimization: 4th International Conference, EMO 2007, Matsushima, Japan, March 5–8, 2007. Proceedings 4*, Springer, 2007, pp. 862–876.
- [41] C.A. Coello Coello, M. Reyes Sierra, A study of the parallelization of a co-evolutionary multi-objective evolutionary algorithm, in: *MICAI 2004: Advances in Artificial Intelligence: Third Mexican International Conference on Artificial Intelligence*, Mexico City, Mexico, April 26–30, 2004. *Proceedings 3*, Springer, 2004, pp. 688–697.
- [42] E. Zitzler, L. Thiele, Multiobjective optimization using evolutionary algorithms—A comparative case study, in: *Parallel Problem Solving from Nature—PPSN V: 5th International Conference Amsterdam*, Springer, 1998, pp. 292–301.
- [43] J.R. Schott, *Fault Tolerant Design Using Single and Multi-Criteria Genetic Algorithms* (Master's Thesis), vol. 37, (1) Massachusetts Institute of Technology, 1995, pp. 1–13.
- [44] Y.-N. Wang, L.-H. Wu, X.-F. Yuan, Multi-objective self-adaptive differential evolution with elitist archive and crowding entropy-based diversity measure, *Soft Comput.* 14 (2010) 193–209.
- [45] G. Reinelt, TSPLIB—A traveling salesman problem library, *ORSA J. Comput.* 3 (4) (1991) 376–384.



Chengxin Wen received the B.S. degree in automation from Beijing Institute of Technology, Beijing, China, in 2017. He is currently working towards the Ph.D. degree in control science and engineering with the School of Automation, Beijing Institute of Technology, Beijing, China. His current research interests include evolutionary computation, multi-objective optimization, and Multi-robot task allocation.



Hongbin Ma received the bachelor's degree from Zhengzhou University, Zhengzhou, China, in 2001, and the Ph.D. degree from the Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing, China, in 2006. From 2006 to 2009, he served as a Research Scientist with the Temasek Laboratories, National University of Singapore, Singapore. He has been a Professor with the School of Automation, Beijing Institute of Technology, Beijing, since 2009. His current research interests include adaptation, learning and recognition, especially applications in robots, self-driving cars, and unmanned aerial vehicle (UAV). Prof. Ma was a recipient of the 13th Huo Ying Dong Young Teacher Award for Higher Education in 2012.